Integrating the RMF Module into grc-toolkit.html

This walks through folding `rmf-module.html` into your existing single-file app as a 13th tab, alongside Evidence Collector, POA&M Tracker, etc.

1. Add the tab button

Find where your existing 12 tab buttons are defined (likely a row of `<button>` elements with `onclick` handlers or a JS array driving them) and add a 13th:

```
<button class="tab-btn" data-tab="rmf">RMF</button>
```

Match whatever class/attribute pattern your other 12 already use — don't introduce a second tab-switching mechanism. If your app switches tabs via a JS function like `showTab('evidenceCollector')`, call the same function with a new tab id, e.g. `showTab('rmf')`, rather than using the `tab-tracker` / `tab-study` / `tab-certs` / `tab-nist` switcher built into the standalone file.

2. Add the panel

Copy the four `<div class="panel" id="panel-...">` blocks (tracker, study, certs, nist) from `rmf-module.html` into a single wrapping panel:

```
<div id="panel-rmf" class="app-panel"> <!-- match your app's panel class -->
  <!-- paste the RMF module's internal 4-tab sub-nav + all 4 panel divs here -->
</div>
```

Keeping the module's own internal sub-tabs (Tracker/Study/Certs/NIST Library) nested inside your single outer "RMF" tab avoids fighting your app's existing tab system — you get a tab-within-a-tab, which is a normal pattern.

3. Namespace the CSS

This is the step most likely to bite you. The module's CSS uses generic class names (`.card`, `.detail-card`, `.task`, `.choice`, `.progress-fill`, etc.) that are likely to collide with your existing 12 tabs' styles.

Prefix every selector in the module's `<style>` block with `.rmf-module`:

```
/* before */
.card{background:var(--paper);...}

/* after */
.rmf-module .card{background:var(--paper);...}
```

Then wrap the pasted panel markup:

```
<div id="panel-rmf" class="app-panel rmf-module">
  ...
</div>
```

A quick way to do the prefixing in bulk: copy the `<style>` block into a scratch file and run a find/replace that prefixes every rule starting with `.` (skip `@media`, `:root`, and `*` selectors — leave those as-is, or better, drop `:root` entirely and inline the CSS variables your existing app already defines if the names collide, e.g. if you already have `--ink` or `--navy` defined elsewhere).

4. Namespace the storage keys

The module uses three `window.storage` keys:

- `rmf-tracker-checks`
- `rmf-quiz-best`
- `rmf-cert-checks`

These are already namespaced with an `rmf-` prefix, so they're unlikely to collide with your other tabs' storage (Evidence Collector, POA&M Tracker, etc.) — but double check none of your existing tabs already use a key starting with `rmf-`.

5. Fonts

The module pulls Source Serif 4 / IBM Plex Sans / IBM Plex Mono from Google Fonts via a `<link>` tag in `<head>`. If your app already loads its own fonts, either:

- merge the `<link>` tags (fine to load multiple font families), or
- swap the module's `font-family` declarations to match your existing app's fonts, so the RMF tab doesn't look visually disconnected from the other 12.

6. Test in isolation first

Before merging, open `rmf-module.html` standalone and click through all four sub-tabs, check a few tracker boxes, run a quiz question, and reload the page to confirm `window.storage` persistence works in your environment. That isolates "did the merge break something" from "did the module ever work."

7. Merge order

Given your dev setup (Cursor + Git, private repo), the safest path:

1. Create a branch (`feature/rmf-module`)
2. Paste in the tab button, panel, CSS (prefixed), and JS in that order
3. Run it locally, click through all 4 sub-tabs
4. Check the browser console for any duplicate `id` warnings — the module uses plain `id` attributes (`taskList` , `studyArea` , `scoreNum` , etc.) which must be unique across your *entire* page, not just within the panel
5. Commit, open a PR against your normal branch even if it's just you reviewing — keeps `docs/architecture.md` accurate if you log it there

Known follow-up (not urgent)

Your existing SCF Explorer / Control Lookup tabs likely already cover control-level detail. The NIST Library tab here is intentionally a summary level — worth linking the two together later (e.g. clicking an 800-53 family in the RMF tab jumps to that family filtered in Control Lookup) rather than maintaining two separate control references.